BioCircuit Studio

A Researcher-Guided Workspace for Organ-on-Chip Planning and EIS Analysis

Submission category: Tool & Platform

Competition: AI4S Open Innovation: AI for Life Science

Author: Ryan Zhang

Version: Existing V2 prototype, packaged October 9, 2026; display names updated October 10, 2026

Abstract

BioCircuit Studio is a software prototype that connects organ-on-chip experiment planning with electrical impedance spectroscopy (EIS) analysis. Its intended users are researchers and students organizing barrier-chip studies who need to keep their experimental question, proposed workflow, electrical model, and sample context together.

The system combines a Streamlit planning interface with an embedded browser-based Fit Lab. Users specify a biological model, experimental condition, separate control, baseline, proposed measurement schedule, and measurement settings. A deterministic selection process chooses an endothelial or epithelial chip template and produces a structured workflow draft. Optional OpenAI API generation and contextual chat extend the template-based descriptions. The plan can be exported as JSON or transferred directly into Fit Lab, where users inspect spectra, adjust an equivalent-circuit model, run multi-start fitting, calculate residual-bootstrap summaries, and export parameters with sample metadata.

The submitted demonstration uses the existing built-in illustrative spectra. Their experimental provenance and generating process are undocumented, and no verified parameter ground truth is available. Evaluation therefore concerns software operation: preservation of planning fields, navigation, import and export, fitting consistency, metadata handling, and offline chart rendering. Scientific accuracy, AI-output quality, and laboratory benefit have not been established.

The contribution is the integration of explicit experimental context, optional AI-assisted planning, inspectable electrical fitting, and researcher-review guidance in one accessible prototype. The local template workflow and Fit Lab run without an API key or private laboratory files. The submission preserves the existing application and makes its capabilities, reproducibility conditions, and limitations explicit.

1. Problem and intended application

This project addresses a practical workflow need: maintaining a clear connection between a proposed barrier-chip experiment and the context used to interpret its electrical measurements. A spectrum's interpretation depends on information such as the biological model, sample condition, control, measurement time, sensing area, and electrical-model assumptions. The application makes these fields visible throughout planning and analysis.

The implemented templates cover an endothelial open-cell EIS configuration and an MDCK membrane-filter configuration. Presets help organize endothelial drug-response, epithelial transport, and barrier-maturation questions. The implemented examples focus on barrier-chip workflows. General performance across organoid types has not been evaluated.

The target outcome is a reviewable draft and an inspectable analysis record. The current software does not control instruments, fabricate a chip, run a biological experiment, or verify a treatment effect.

2. Architecture and implemented workflow

Stage	Component	Implemented output
Planning	Streamlit Workshop and `workflow_context.py`	Editable research question, condition, control, baseline, proposed schedule, settings, and metadata plan
Draft generation	`microfluidic_agent.py`	Selected template, reasons, proposed configuration, workflow, and analysis guidance
Optional AI	External API generation and section chat	Additional draft descriptions and responses conditioned on supplied context
Handoff	JSON export/import or direct transfer	Planning context supplied to Fit Lab
Fitting	Embedded `eis_simulator_v2.html`	Interactive plots, fitted parameters, residual summaries, optional bootstrap
Export	Browser CSV export	Parameters, uncertainty fields when available, and per-spectrum metadata

The Workshop separates the biological sample baseline from the cell-free reference and the experimental condition from its control. Its checklist reports whether information has been entered. It does not assess whether a protocol is scientifically adequate. Workflow outputs carry the status "draft awaiting researcher review".

Fit Lab maintains sample identifiers, condition, time point, biological replicate, chip identifier, operator/date, biological model, experimental condition, control, active area, settings, and provenance per spectrum. Planned information can carry over from the Workshop; actual sample identifiers and time points remain empty until entered. The user can inspect and edit context before exporting.

The interface includes suggested downstream deliverables, such as time-course summaries and a design report. The actual download features in this version are workflow JSON and fitting CSV. Automatic fit-quality classification remains labelled as proposed. Researcher review is advisory, with no approval record or enforced sign-off.

3. Planning logic and AI contribution

3.1 Local template mode

The application normalizes the request and selected biological components into keywords. It scores two templates using matching cell-model terms and indicators for endothelial or membrane-based workflows. Longitudinal and perturbation terms add context. The highest-scoring template is selected, with an endothelial fallback when no strong match is found. The generated package contains the selection reasons, architecture, ports, electrode suggestions, assembly guidance, measurement plan, and analysis setup.

This mode is deterministic and requires no external model. Its scope is constrained by the two hand-authored templates. Template recommendations have not been independently validated as complete experimental protocols.

3.2 Optional AI generation and chat

When enabled, the existing OpenAI API path sends the user's request, selected template, draft configuration, analysis guidance, and selected source descriptions to an external language model. The requested response contains structured Assay Summary, Build, Operate, and Analyze sections, which are merged into the base package. Section-specific chat receives relevant package context and conversation history.

The prompt asks the model to preserve the experimental condition, control, baseline, and proposed schedule; acknowledge calibration needs; and avoid unsupported dimensions, biological claims, sample identifiers, or acceptance thresholds. These instructions express intended behavior. A live evaluation of whether model outputs consistently follow them has not been conducted.

The source code's default model string is `gpt-5.5`. Users supply their own credentials and choose a model available to their account. API cost and availability depend on that account. The supplied software checks use a stub response; no live API calls or model-quality comparisons were made during submission preparation.

Two source records remain in the code, named `2023 Mai XC - Endo` and `2023 June LF MDCK MF`. Their underlying PDFs are excluded from the public package. The existing code reports them as missing and continues with its templates. The API receives source titles, roles, and short scope descriptions. Full-text PDF reading, retrieval-augmented generation, and page-level evidence tracing are not implemented.

4. EIS model and numerical methods

4.1 Equivalent circuit

The active browser Fit Lab uses a series combination of bulk resistance, a constant-phase element (CPE), and a parallel resistance-capacitance cell-layer term:

```
Z(omega) = R_bulk + 1 / [A (j omega)^n]
           + R_cell / [1 + j omega C_cell R_cell]

omega = 2 pi f;  j^2 = -1;  A = 10^a
```

Here `R_cell` is labelled CLER, and `C_cell` is entered as CLEC in microfarads before conversion to farads. The fitted vector is `(R_bulk, CLER, CLEC, n, a)`. The model's biological interpretation depends on the selected device, calibration, and experimental context.

4.2 Fitting objective and optimizer

Fit Lab minimizes a magnitude-normalized squared residual across real and imaginary components:

```
L(p) = sum_i { [Re(Z_model,i) - Re(Z_data,i)]^2
            + [Im(Z_model,i) - Im(Z_data,i)]^2 }
            / (|Z_data,i| + 10^-10)^2
```

The browser implementation uses five initial parameter sets and Nelder-Mead search, allowing up to 4,000 iterations per start. Parameters outside the bounds receive a large penalty. The lowest-cost valid candidate is retained. This is a local search without a guarantee of a unique or global optimum.

Parameter	Browser Fit Lab range	Unit
Bulk resistance	30-500	ohm
CLER	0.5-2,000	ohm

Parameter	Browser Fit Lab range	Unit
CLEC	0.01-30	microfarad
CPE exponent n	0.35-1	dimensionless
log10(A)	-11 to -2	logarithm of the CPE coefficient in the model convention

The app also contains an auxiliary Python routine using SciPy least squares with different bounds. The Workshop's analysis guidance reflects that broader routine. The interactive Fit Lab uses the JavaScript optimizer and bounds above; importing a workflow does not replace them. The distinction matters when reproducing displayed results.

4.3 Outputs, plots, and uncertainty

For sensing area `S` in cm², the app calculates `TEER = CLER × S`, area-normalized capacitance `CLEC / S`, and characteristic frequency `1 / (2 pi CLER CLEC × 10^-6)`. These are model-derived quantities. The software does not implement a validated cell-free subtraction protocol.

The display provides Bode magnitude, Bode phase, and Nyquist plots. The existing Nyquist view displays frequencies below 10 kHz; fitting uses the full selected spectrum. Residual summaries include unweighted RMS in ohms and the normalized weighted sum of squared residuals. That latter value is an objective score without calibrated measurement-error variances; it is not a statistical goodness-of-fit probability.

The optional bootstrap independently resamples real and imaginary residuals across frequency, adds them to the fitted curve, and refits from the current estimate with a shorter search. It displays parameter standard deviations, percentile intervals, and a magnitude band. It uses unseeded browser randomness. Frequency correlation, real/imaginary covariance, model misspecification, and interval coverage have not been validated. Bootstrap intervals should be treated as exploratory numerical summaries.

5. Data, provenance, and public packaging

The built-in demonstration comprises 12 spectra at 65 frequencies from 5 Hz to 100 kHz. The software assigns labels from 0 to 33 h in 3 h steps. Experimental provenance, a synthetic generating process, ground-truth electrical parameters, and biological sample metadata are undocumented. The values are described throughout this submission as illustrative demo data. The project owner authorized their public inclusion on October 9, 2026.

All separate laboratory datasets, private PDFs, earlier result exports, notebooks, personal settings, logs, and unrelated workspace files are excluded. No training data or learned model weights are required. The public package contains the application source with updated display names, built-in data arrays, bundled Chart.js, dependency specifications, documentation, and reproducible software checks.

The matrix CSV importer expects a frequency row followed by four-row spectrum blocks containing magnitude, real impedance, imaginary impedance, and phase. It requires numeric rectangular data and a monotonic positive frequency axis. The active browser parser uses real and imaginary rows and maps imaginary values to a negative magnitude. This convention limits handling of inductive spectra. Uploaded groups are retained and must be identified by the researcher.

Provenance choices distinguish built-in demo data, unverified uploads, and user-confirmed measured data. The measured-data selection records a user assertion. Authentication against experimental records is outside the application's capabilities.

6. Software verification and observed results

Submission preparation exercised the packaged application using its existing tests, adapted to the absence of private files. No new scientific benchmark or biological experiment was introduced.

Check	Result	What it establishes
Python workflow tests	4 passed	Plan/JSON round trip, unknown metadata, invalid imports, stubbed API contract, and Workshop navigation
Browser checks	18 passed	Existing demo workflow, data/context handoff, fitting/export consistency, bootstrap completion, and interface behavior
Source integrity	7 unchanged; 1 display-only update	The Streamlit file differs only in four display-name strings; other application/dependency files match the V2 source digests

Browser verification uses the built-in demo and reimports two of those spectra through the CSV interface. It checks preservation of metadata, invalid-input handling, plot modes, workflow reload, and chart operation with networking disabled. Entered conditions, schedules, and sample identifiers are test fixtures. Their presence in exported files does not establish that an experiment occurred.

The verification environment uses Python 3.13.5, Streamlit 1.58.0, NumPy 2.4.6, pandas 3.0.3, SciPy 1.17.1, and OpenAI SDK 2.44.0. The browser checks use Playwright and local headless Chrome. Tests were run against the separate public package using the existing local Python environment. A fresh dependency installation and cross-platform operation remain unverified.

These results support reproducibility of the demonstrated software interactions in the stated environment. No parameter-recovery accuracy, drug-response performance, AI-benefit comparison, user-time saving, expert protocol score, or confidence-interval coverage is reported.

7. Reproduction

Install Python 3.13, create a virtual environment, install `requirements.txt`, and run the application from the repository root:

```
python -m streamlit run software/microfluidic-eis-agent/streamlit_app.py
```

Leave the API key empty for the local demonstration. Select a Workshop preset, enter a proposed baseline and schedule, generate a draft, inspect its views, and use "Open Workflow in Fit Lab". Run Auto-fit on the selected built-in spectrum, optionally run bootstrap, inspect the three plot modes, and download the fitting CSV. Save exports before navigating away.

Run `python verification/test_workshop.py` and `python verification/check_integrity.py` for the packaged Python and integrity checks. The README provides optional browser-test setup and the complete input/output contract. A standalone Fit Lab can also be opened from its HTML file with the bundled chart library alongside it.

No paid service or private dataset is needed for the local demonstration. Reproducing AI generation requires separate API access, and its output may vary with the selected model and service behavior.

8. Limitations and practical contribution

The system's present contribution is an integrated interface: experimental context accompanies a workflow draft and remains available during electrical fitting and result export. The templates, model equations, parameters, and residual summaries provide concrete points for researcher inspection. This may support teaching, workflow discussion, and early research preparation; those benefits have not been measured in a user study.

The main limitations are the narrow template scope, unavailable source PDFs, unverified demo provenance, lack of biological validation, absence of a live AI-quality evaluation, and exploratory numerical uncertainty estimates. Electrical parameters may be poorly identifiable, sensitive to starting values, or constrained by the existing bounds. A plausible-looking fit alone does not establish biological meaning.

The application also lacks instrument integration, an enforced researcher approval process, persistent Fit Lab sessions, and automatic fit-quality classification. Future work could evaluate these needs with domain experts and authorized experimental data. Such work is outside this submission's completed scope.

9. Sources and software

The primary technical evidence for this report is the submitted source and its verification outputs:

1. `software/microfluidic-eis-agent/streamlit_app.py`: interface, navigation, handoff, and API configuration.

2. `software/microfluidic-eis-agent/microfluidic_agent.py`: templates, reference records, optional API requests, and auxiliary Python numerical methods.

3. `software/microfluidic-eis-agent/workflow_context.py`: planning fields and JSON context validation.

4. `software/microfluidic-eis-agent/fitlab_assets/eis_simulator_v2.html`: demo arrays, active browser fitting model, optimizer, bootstrap, plots, and CSV export.

5. `verification/VALIDATION.md` and associated result summaries: packaging checks and their scope.

The project uses Streamlit, NumPy, pandas, SciPy, the OpenAI Python SDK, and bundled Chart.js 4.4.1. Python dependencies are installed separately under their own licenses; the Chart.js MIT notice is retained. `THIRD_PARTY_NOTICES.md` records versions, sources, and license information. No additional license for the original project code is granted in this repository.

Competition requirements were consulted at https://www.kaggle.com/competitions/ai-4-s-open-innovation-artificial-intelligence-for-life-scien/overview/url on October 9, 2026. The competition page informs submission format; it supplies no validation of this project.